IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Samalka Anandagoda
06/16/2025

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
  - Data collection via API and Web Scraping
  - Exploratory data analysis (EDA) with SQL
  - Exploratory data analysis (EDA) with Data visualization
  - Interactive maps with Folium
  - Building a Dashboard application with Plotly Dash
  - Predictive Analysis
- Summary of all results
  - Exploratory Data Analysis results
  - Interactive maps and Dashboard
  - Predictive results

Introduction

- Project background and context
  - In this project we predict if the Falcon 9 first stage will land successfully. The advertised cost for a Falcon 9 rocket launches is 62 million dollars based on SpaceX website. However, other providers estimate a cost of 165 million dollars for each rocket launch. The price difference is attributed to the fact SpaceX can reuse the first stage and therefore save a significant amount of money. Hence by determining if the first stage will land, we can determine the cost of a launch. Using these predictions an alternate company can bid against SpaceX for a rocket launch. This information can then be used by an alternate company if it wants to bid against SpaceX for a rocket launch.

- Problems you want to find answers
  - What are the main attributes that govern a successful landing or a failed landing?
  - Are there any common characteristics in all successful landings?
  - Does payload mass play a role in the successful or failed landings?

Section 1

Methodology

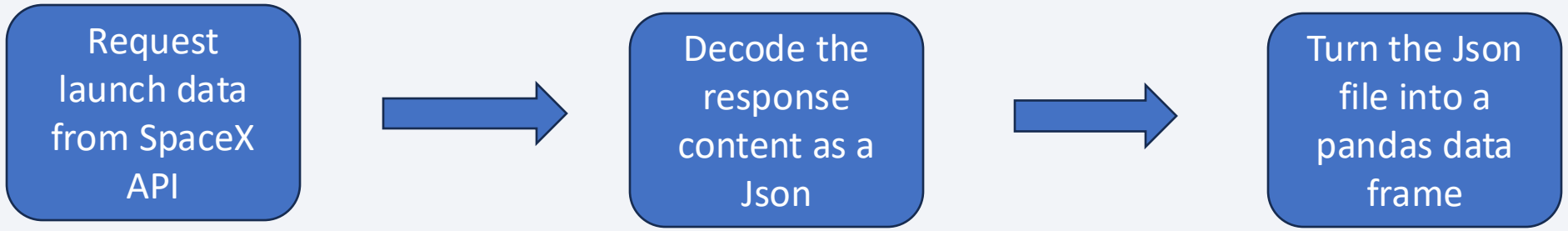
Methodology

Executive Summary

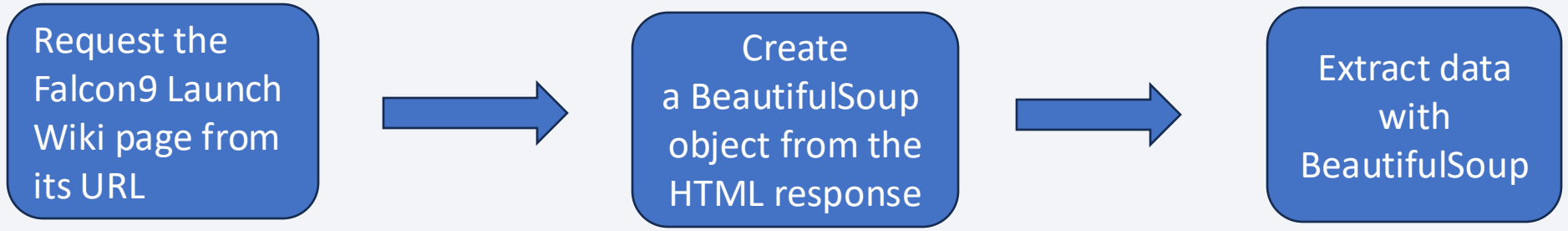
- Data collection methodology:
  - SpaceX REST API
  - Web Scrapping from Wikipedia
- Perform data wrangling
  - Determining training labels
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - How to build, tune, evaluate classification models

Data Collection

- Data are collected from SpaceX REST API and Web Scrapping Wikipedia
  - SpaceX rocket launch data are obtained from SpaceX API using the following URL: https://api.spacexdata.com/v4/launches/past



  - Information such as launches, landing and payload weight are obtained by Web Scrapping Wikipedia using the URL: https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922



Data Collection – SpaceX API

1. Getting the response from SpaceX API

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
response = requests.get(spacex_url)
```

2. Converting the response to a JSON file

```
data = response.json()
```

3. Converting the JSON file into a pandas data frame

```
df = pd.json_normalize(data)
```

4. Creating a data frame including only the following columns using a dictionary

```
launch_dict = {'FlightNumber': list(df['flight_number']),
'Date': list(df['date']),
'BoosterVersion':BoosterVersion,
'PayloadMass':PayloadMass,
'Orbit':Orbit,
'LaunchSite':LaunchSite,
'Outcome':Outcome,
'Flights':Flights,
'GridFins':GridFins,
'Reused':Reused,
'Legs':Legs,
'LandingPad':LandingPad,
'Block':Block,
'ReusedCount':ReusedCount,
'Serial':Serial,
'Longitude': Longitude,
'Latitude': Latitude}
```

5. Filtering the data frame to only include Falcon 9 launches

```
data_falcon9 = df_new[df_new['BoosterVersion']!='Falcon 1']
```

6. Exporting data to file

```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

- GitHub URL of the completed SpaceX API calls notebook: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/jupyter-labs-spacex-data-collection-api-v2.ipynb

Data Collection - Scraping

1. Request the HTML page and get a response object

```
response = requests.get(static_url)
```

2. Create BeautifulSoup Object

```
soup = BeautifulSoup(response.text, 'html.parser')
```

3. Extract all column names from the HTML table header

```
html_tables = soup.find_all('table')
```

4. Create a dictionary with extracted column names

```
launch_dict= dict.fromkeys(column_names)
```

5. Filling data into the empty arrays

Eg:

```
bv=booster_version(row[1])
if not(bv):
    bv=row[1].a.string
launch_dict['Version Booster'].append(bv)
```

6. Converting the dictionary to a Data Frame

```
df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
```

7. Export data

```
df.to_csv('spacex_web_scraped.csv', index=False)
```

- GitHub URL of the completed Web Scraping notebook: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/jupyter-labs-webscraping.ipynb

Data Wrangling

1. Read the SpaceX dataset onto a data frame

2. Identify missing values in each attribute and identify numerical and categorical columns

```
df.isnull().sum()
df.dtypes
```

3. Calculate number of launches on each site

```
df['LaunchSite'].value_counts()
```

4. Calculate number and occurrence of each orbit

```
df['Orbit'].value_counts()
```

5. Calculate the number and occurrence of mission outcome of the orbits

```
landing_outcomes = df['Outcome'].value_counts()
```

```
landing_outcomes
```

```
Outcome
True ASDS      41
None None      19
True RTLS      14
False ASDS      6
True Ocean      5
False Ocean     2
None ASDS       2
False RTLS      1
Name: count, dtype: int64
```

6. Create a landing outcome label from Outcome column.

```
# landing_class = 0 if bad_outcome
# landing_class = 1 otherwise
landing_class = [0 if outcome in bad_outcomes else 1 for outcome in df['Outcome']]
```

7. Create another column called class which shows the landing outcome. 0 or 1 based on a successful landing or not.

```
df['Class']=landing_class
```

- GitHub URL of the completed Data Wrangling notebook: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/labs-jupyter-spacex-Data%20wrangling-v2.ipynb

EDA with Data Visualization

Scatter Plots:
Shows the relationship between variables

- Flight number vs Pay load mass
- Flight number vs Launch Site
- Payload vs Launch Site
- Orbit vs Flight Number
- Payload vs Orbit

Bar Plot:
Shows the relationship between a numerical variable and a categorical variable

- Success rate by Orbit

Line Graph:
Shows how a variable changes with respect to another variable. Line graphs show trends and can be used to make predictions for unseen data

- Success rate vs Year

- GitHub URL of the completed EDA with Data Visualization notebook: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/jupyter-labs-eda-dataviz-v2.ipynb

EDA with SQL

- Data was read into a data frame and then converted into a SQL table called SPACEXTABLE
- Display the names of the unique launch sites in the space mission
- Display 5 records where launch sites begin with the string 'CCA'
- Display the total payload mass carried by boosters launched by NASA (CRS)
- Display average payload mass carried by booster version F9 v1.1
- List the date when the first succesful landing outcome in ground pad was acheived
- List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000
- List the total number of successful and failure mission outcomes
- List all the booster versions that have carried the maximum payload mass
- List the records which will display the month names, failure landing outcomes in drone ship, booster versions, launch Site for the months in year 2015
- Rank the count of landing outcomes between the date 2010-06-04 and 2017-03-20, in descending order

- GitHub URL of the completed EDA with SQL notebook: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/jupyter-labs-eda-sql-coursera_sqllite.ipynb

Build an Interactive Map with Folium

- Folium Map object was created with an initial center location to be NASA Johnson Space Center at Houston, Texas
- Blue circle added to show the Johnson Space Center on the map as well as a text showing the name
- Blue circles along with text labels are added to mark all the launch sites on the map
- Successful and failed launches for each Launch Site was marked on the map using different colors. Green is used to indicate successful launches and red to indicate failed launches
- Distances between launch sites to other key locations like nearby cities, railroad and highways are marked on the map
- These markers are added to the map to better visualize where the launch sites are located, their proximity to cities, railroads and highways. This is essential in planning future launches and transporting rockets to launch sites.

- GitHub URL of the completed Interactive map with Folium notebook: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/lab-jupyter-launch-site-location-v2.ipynb

Build a Dashboard with Plotly Dash

- Dashboard contains a dropdown option where the user can select their desired Launch Site
- Pie charts are shown containing information such as the percentage of successful landings and failed landing based on the dropdown selection of the user.
- Rangeslider option allows the user to select a payload mass range and view information pertaining to the selected range
- Scatter plot shows Success vs Payload mass graph in the range selected by the user using Rangeslider option

- GitHub URL of the completed Plotly dash app: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/spacex_dash_app.py

Predictive Analysis (Classification)

1. Data preparation
   Load Data, Standardize the data and split the data into training and testing sets.

2. Model Building
   Select the Machine Learning Algorithm
   Create a GridSearchCV object using a dictionary of parameters
   Fit the model using the training data set

```
parameters ={"C":[0.01,0.1,1],'penalty':['l2'], 'solver':['lbfgs']}# l1 lasso l2 ridge
lr=LogisticRegression()

# Create GridSearchCV object with 10-fold cross-validation
logreg_cv = GridSearchCV(lr, parameters, cv=10)

# Fit the model on training data
logreg_cv.fit(X_train, Y_train)
```

3. Model Evaluation
   Get the hyperparameters for each model
   Calculate the accuracy of the model using the test dataset
   Plot the confusion matrix for the test dataset

```
yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

4. Model Comparison
   Calculate the accuracy of each model and compare them to each other
   The model with the highest accuracy value is chosen to be the best

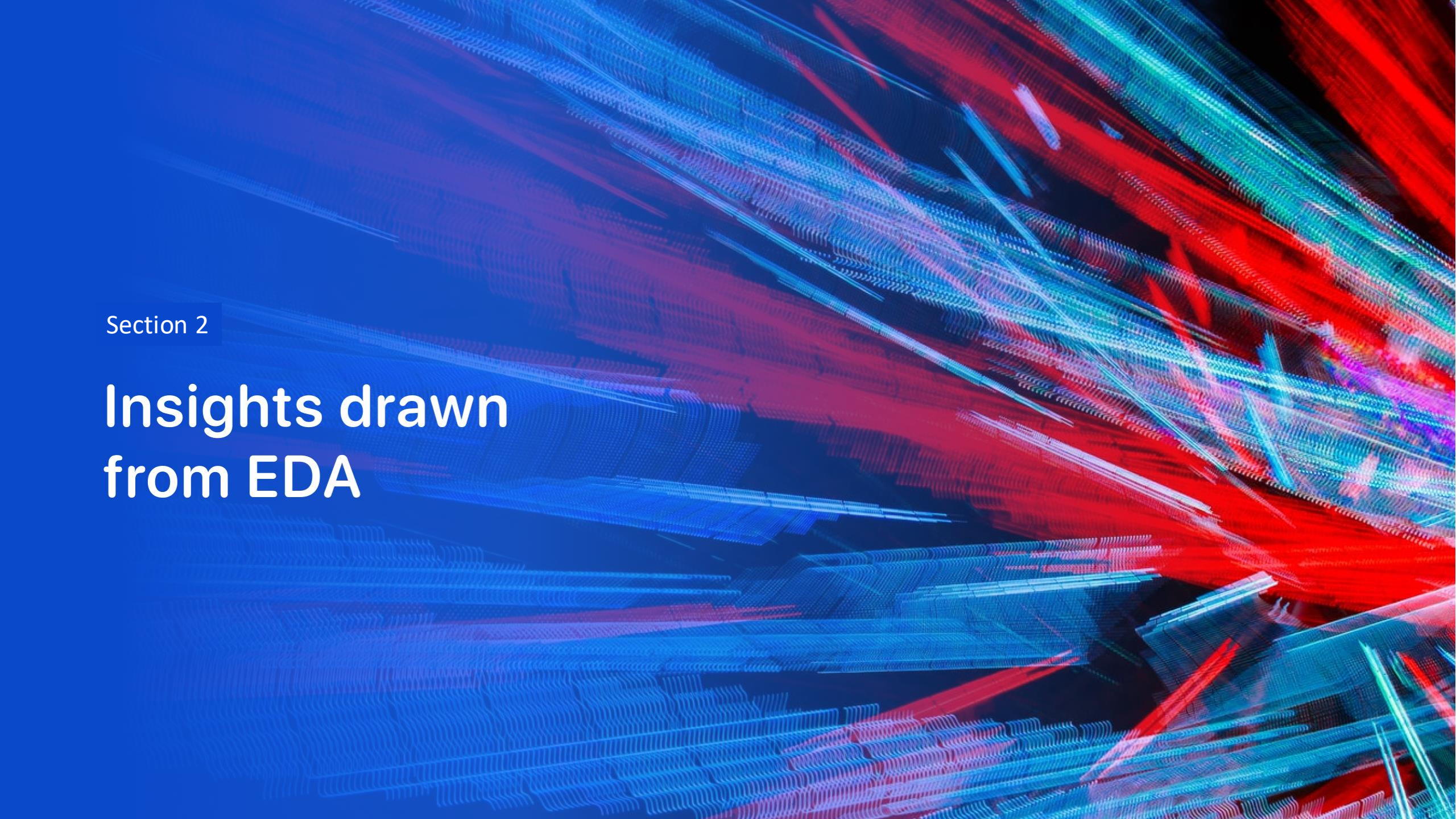
```
Logreg: 0.8464285714285713
svm: 0.8482142857142856
Tree: 0.875
Knn: 0.8482142857142858

Decision Tree Classifier performs best
```

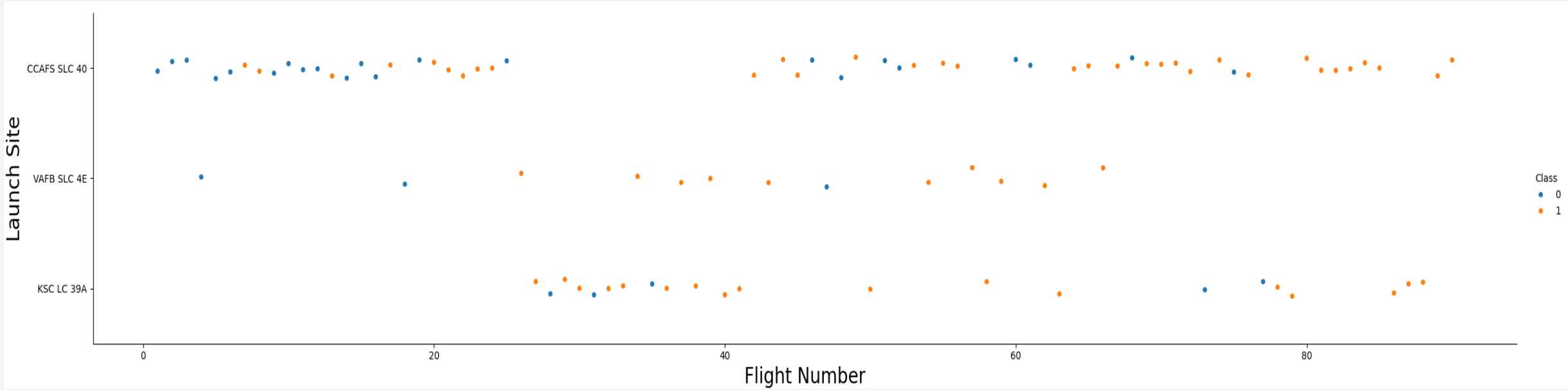
- GitHub URL of the completed Predictive Analysis notebook:
  https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/blob/master/SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb

Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

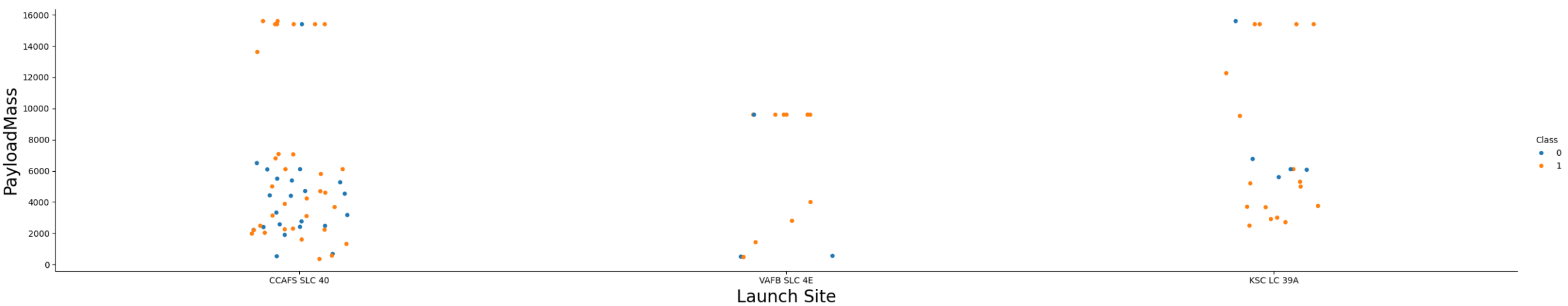

Section 2
Insights drawn from EDA

Flight Number vs. Launch Site



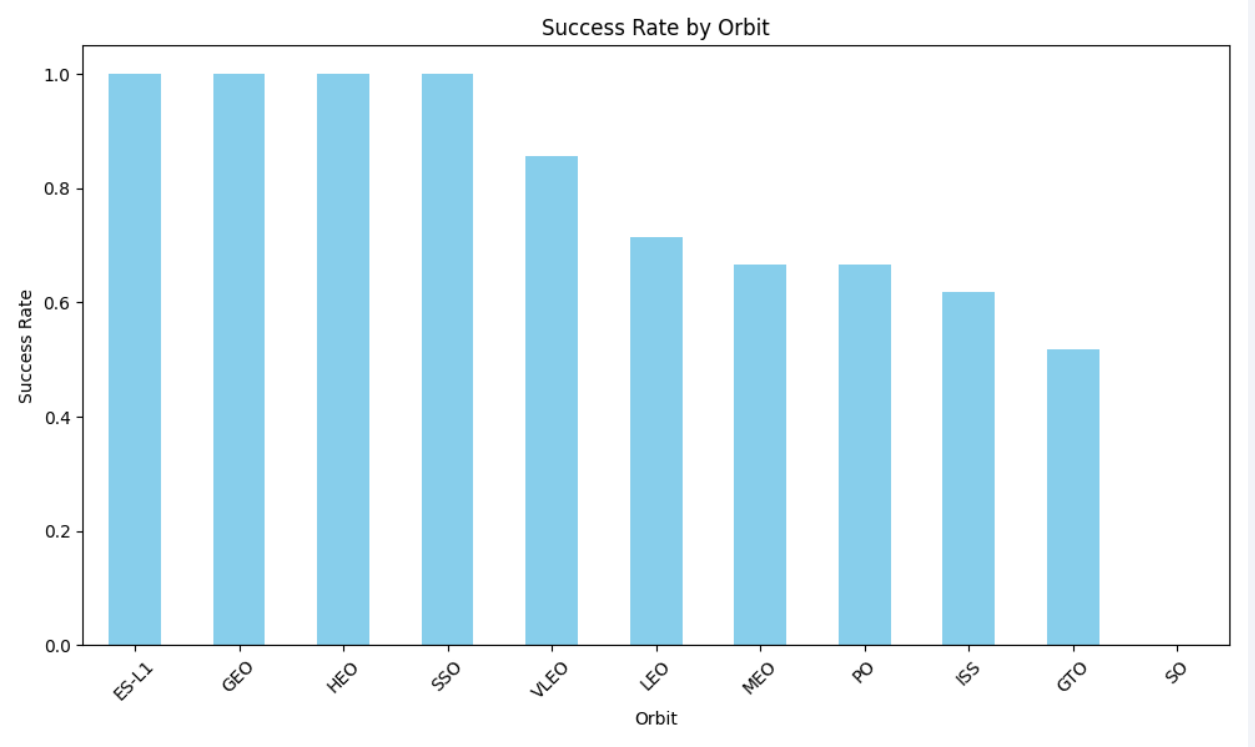
- The success rate in all the Launch sites are higher. The Launch Site CCAFS SLC 40 has the highest number of successful flights.

Payload vs. Launch Site



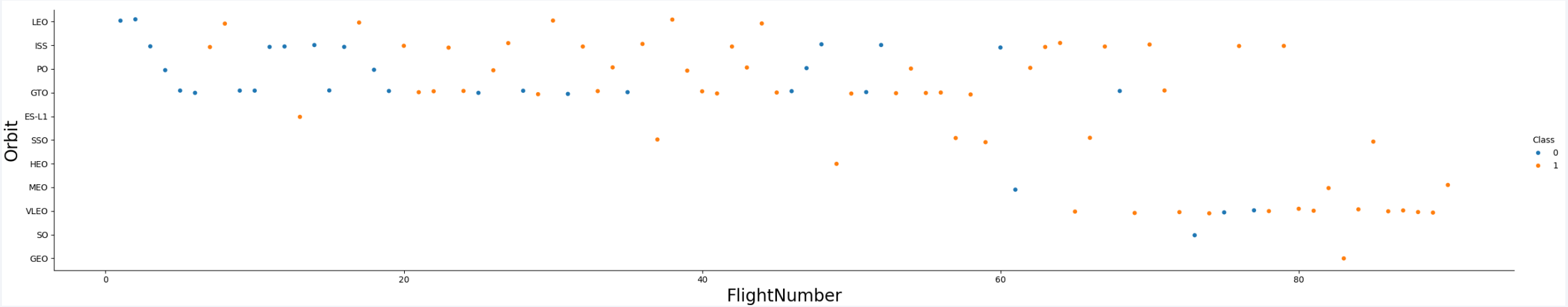
- Based on the plot some Launch Sites have launched rockets with higher payloads. A higher payload doesn't seem to create a failed landing according to the graph as there are many successful landings with higher payloads.

Success Rate vs. Orbit Type



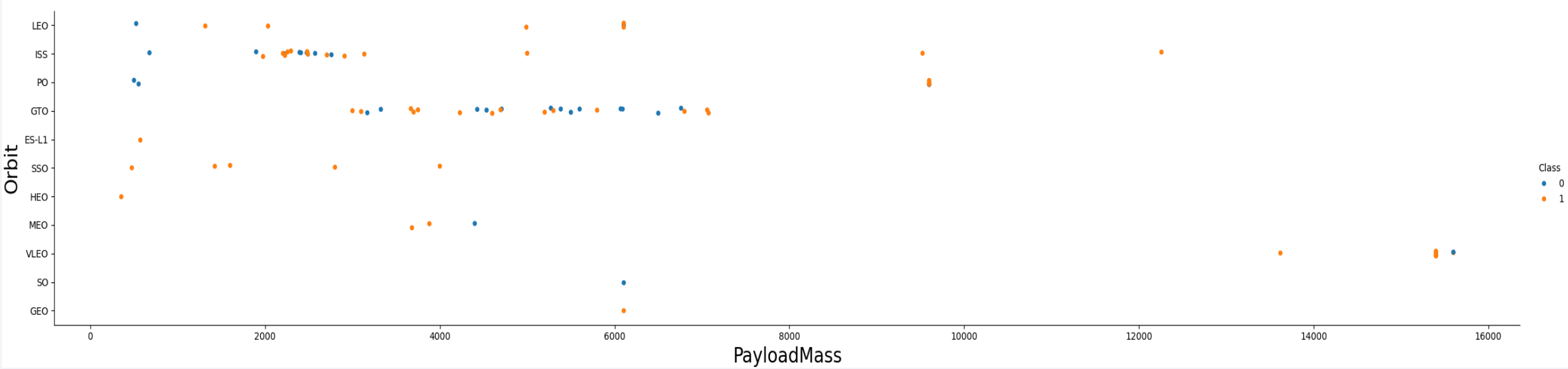
- The success rates for orbits ES-L1, GEO, HEO, SSO are at 100%. The Success rate for orbit SO is very low.

Flight Number vs. Orbit Type



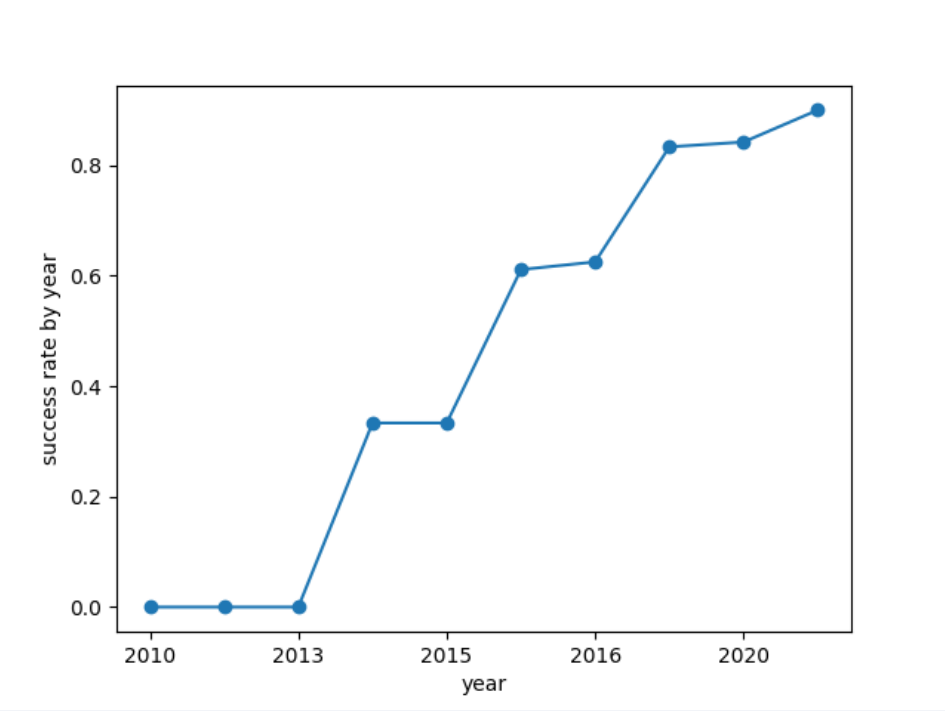
- For the LEO orbit, successful landings increase as the Flight number increase. In general, successful landings increase as the Flight Number increases.

Payload vs. Orbit Type



- For LEO orbit, as the Payload Mass increases, the successful landings also increases. The same trend does not apply to other orbits.
- In orbits MEO, VLEO as the payload mass increases, the successful landings decrease.

Launch Success Yearly Trend



- The success rate of landings are increasing with time since 2013.

All Launch Site Names

SQL query to find all Launch Site Names:

```
%sql select distinct "Launch_Site" from SPACEXTBL
```

Results:

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Explanation:

The ‘distinct’ keyword is used to select only the unique names if the Launch Sites from the SPACEXTBL table. This will eliminate Launch Sites with the same name.

Launch Site Names Begin with 'CCA'

SQL query to find Launch Site Names Beginning with ‘CCA’:

```
%sql select * from SPACEXTBL where "Launch_site" like 'CCA%' limit 5
```

Results:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Explanation:

Selected all the data from the table where the name of the Launch Site begins with ‘CCA’.
This is done using the keywords ‘where’ and ‘like’. Only 5 data entries were shown here by using the keyword ‘limit’

Total Payload Mass

SQL query to find the total payload mass carried by NASA:

```
%sql select sum(PAYLOAD_MASS__KG_) from SPACEXTBL where Customer like '%NASA (CRS)%'
```

Result:

sum(PAYLOAD_MASS__KG_)
48213

Explanation:

The query return the total payload mass carried by NASA. This is done by the use of keywords such as ‘where Customer like’ as shown in the above query.

Average Payload Mass by F9 v1.1

SQL query to find the total payload mass carried by booster version F9 v1.1:

```
%sql select avg(PAYLOAD_MASS__KG_) from SPACEXTBL where Booster_Version like 'F9 v1.1'
```

Result:

avg(PAYLOAD_MASS__KG_)
2928.4

Explanation:

The query return the average payload mass carried by booster version F9 v1.1.

First Successful Ground Landing Date

SQL query to find the first Successful Ground Landing Date:

```
%sql select Min(Date) from SPACEXTBL where Landing_Outcome like 'Success (ground pad)'
```

Result:

Min(Date)
2015-12-22

Explanation:

The Query return the date of the first Ground Landing date. This is achieved by the use of keywords such as ‘Min(Date)’ and Landing_Outcome like ‘Success (ground pad)’

Successful Drone Ship Landing with Payload between 4000 and 6000

SQL query:

```
%sql select Booster_Version from SPACEXTBL where Landing_Outcome like 'Success (drone ship)' and PAYLOAD_MASS__KG_ between 4000 and 6000
```

Result:

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Explanation:

The Query returns the booster versions where the landing outcome is Success (done ship). This is achieved using the 'where' and 'like' keywords to filter the data from the SPACEXTBLE table.

Total Number of Successful and Failure Mission Outcomes

SQL query:

```
%sql select count(*) from SPACEXTBL where Landing_Outcome like 'Success%'
```

Result:

count(*)
61

SQL query:

```
%sql select count(*) from SPACEXTBL where Landing_Outcome like 'Failure%'
```

Result:

count(*)
10

Explanation:

The queries above returns the total number of Successful and Failed landing outcomes.
This is done using keywords such as ‘WHERE’ and ‘LIKE’ to filter the data from the table.

Boosters Carried Maximum Payload

SQL query:

```
%sql select Booster_Version, PAYLOAD_MASS__KG_  FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_  = (SELECT MAX(PAYLOAD_MASS__KG_ ) FROM SPACEXTBL)
```

Results:

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

Explanation:

The query above returns the names of all the Booster versions that have carried the maximum payload. A subquery was used to filter the results containing only the maximum payload.

2015 Launch Records

SQL query:

```
%sql SELECT substr(Date, 6, 2) AS Month, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTBL WHERE Landing_Outcome LIKE '%Failure (drone ship)%' AND substr(Date, 1, 4) = '2015'
```

Results:

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Explanation:

The query shows the Booster Version and Launch Site for failed landing in drone ships that have taken place in the year 2015. substr(Date, 6, 2) shows the month and substr(Date, 1, 4) shows the year.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

SQL query:

```
%sql SELECT Landing_Outcome, COUNT(*) AS Total FROM SPACEXTBL where Date between '2010-06-04' and '2017-03-20' group by Landing_Outcome order by Total DESC
```

Results:

Landing_Outcome	Total
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

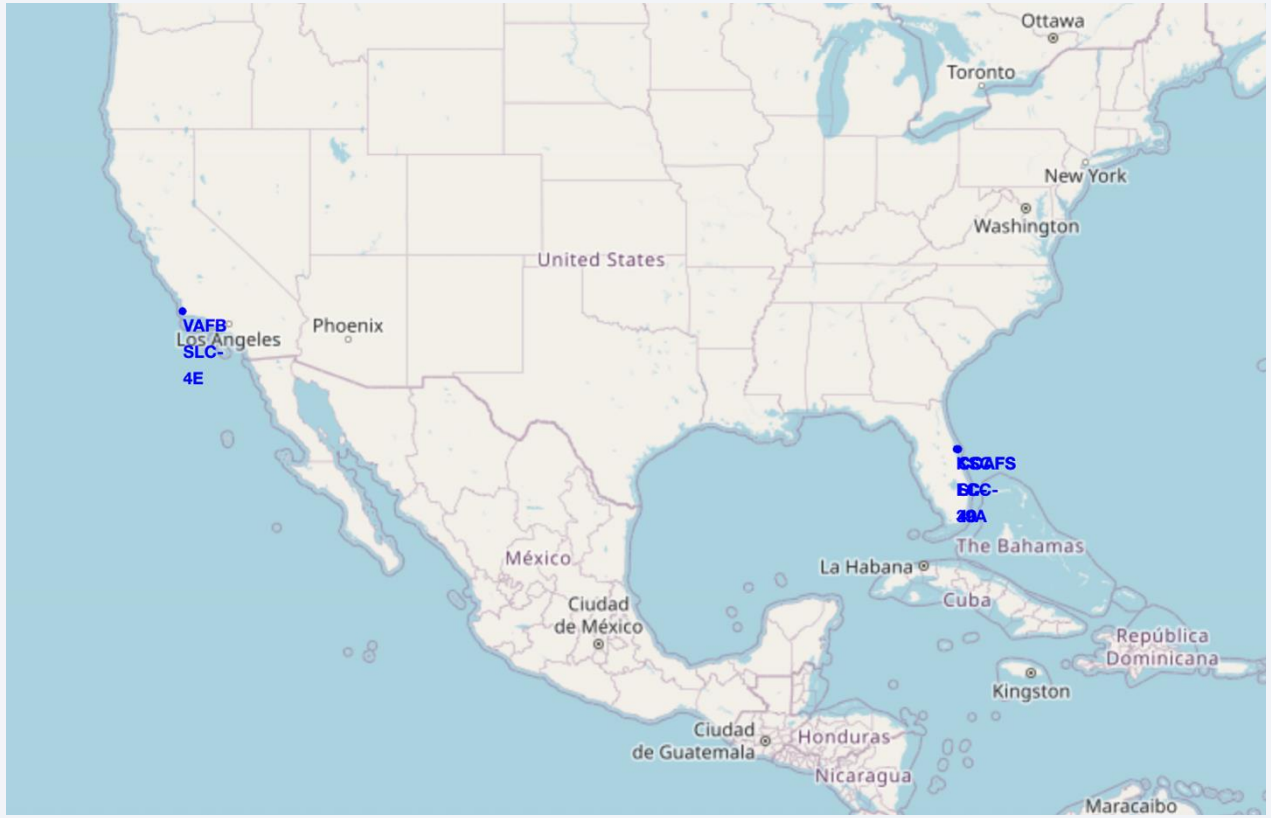
Explanation:

The query shows the total count of various landing outcomes from 2010-06-04 to 2017-03-20. It then ranks the count in the descending order. The GROUP BY keyword groups the results by landing outcomes and ORDER BY TOTAL DESC arranges the total count in descending order.

Section 3

Launch Sites Proximities Analysis

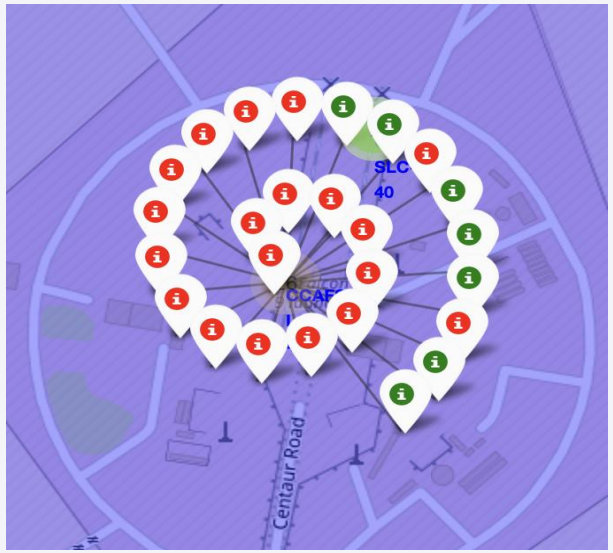
Folium Map SpaceX Launch Sites



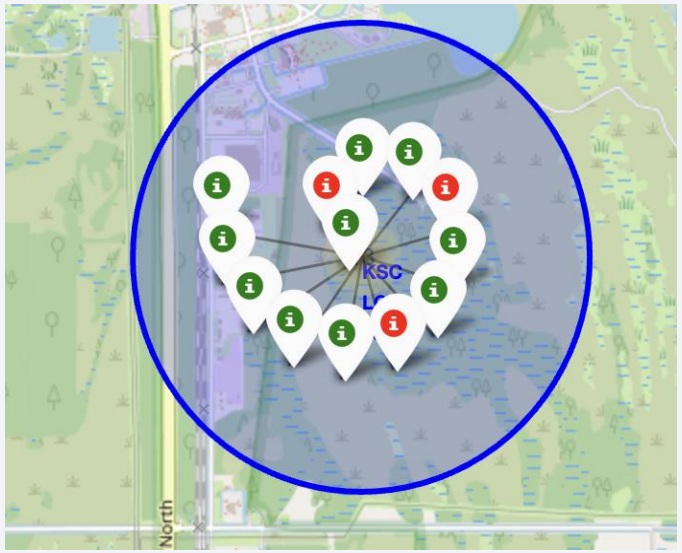
- The map shows the locations of the Launch Sites. We see that they are all located close to the coast.

Successful and Failed Launches

CCAFS LC-40



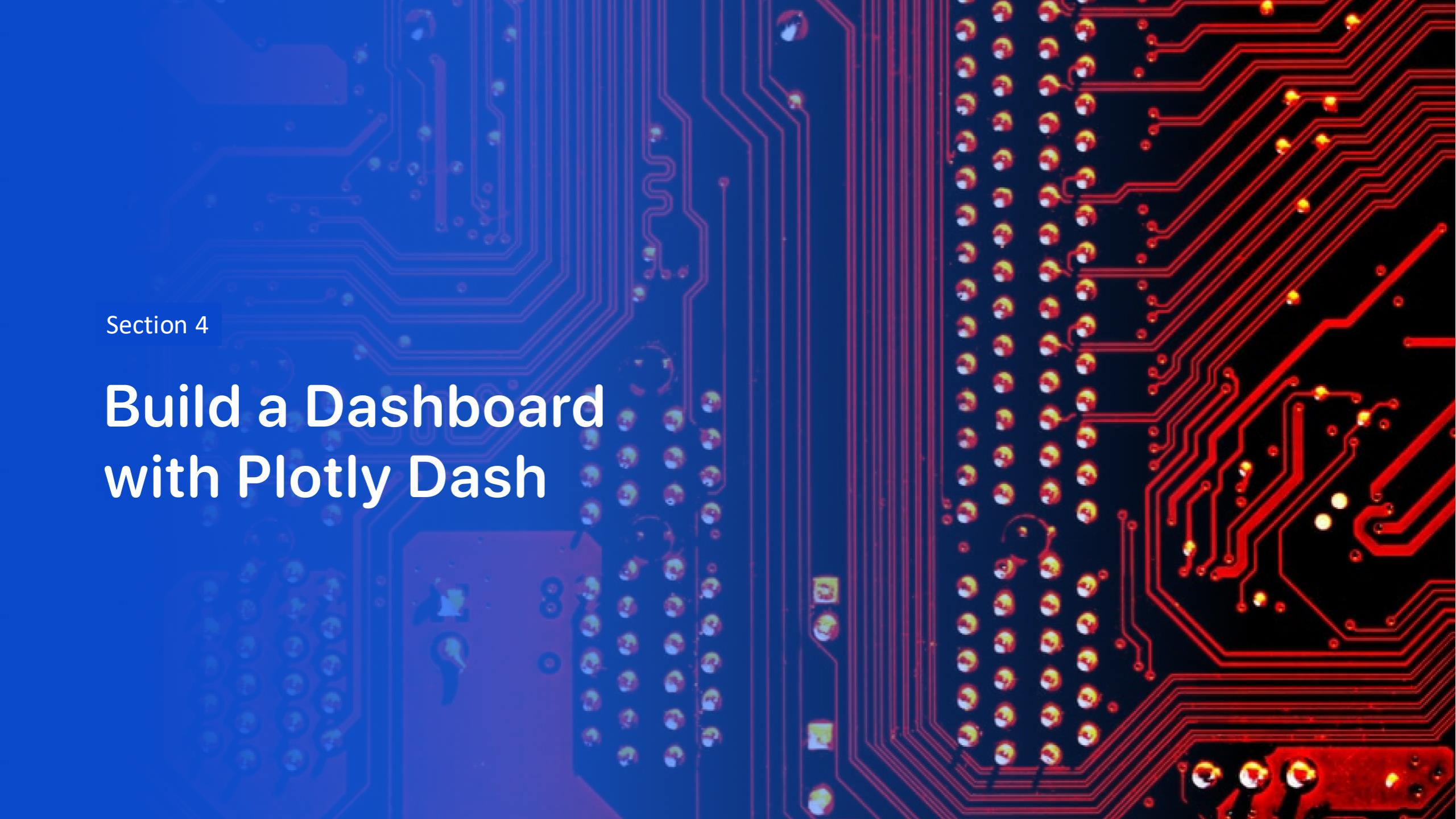
KSC LC-39A



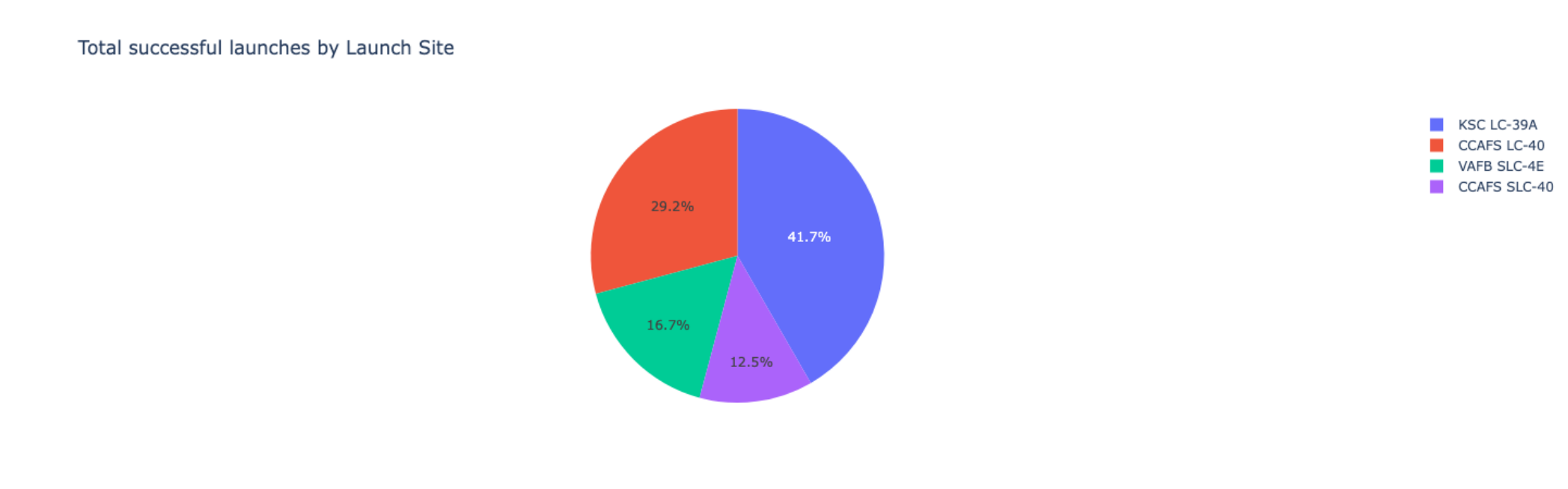
- The maps above show the successful and failed landing of two Launch Sites. The red markers denotes failed landings while green markers denote the successful landings.

Launch Site CCAFS SLC-40 and its Proximities

- We can infer many information by indicating the distances from the Launch site to railroads, coastline, highways and nearby cities on the map.
- This information can be used in making decisions regarding transportation of rockets (rail roads used for transporting rocket parts) and risk assessment (having a city nearby would be a high risk).

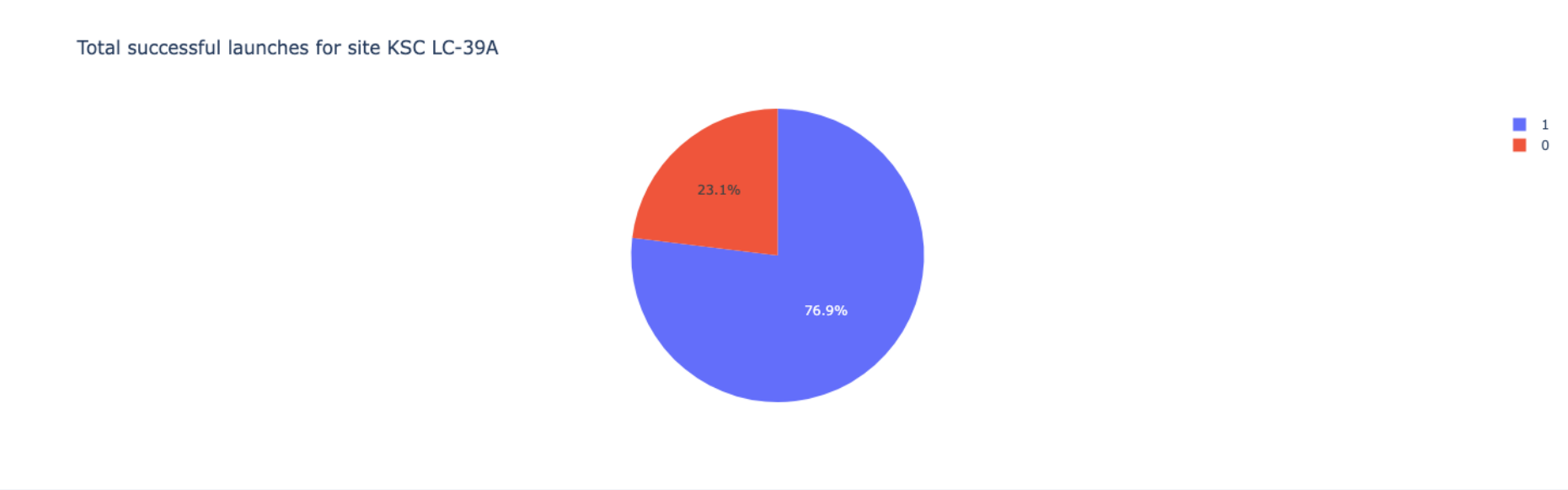

Section 4
Build a Dashboard with Plotly Dash

Dashboard : Total Success by Launch Site



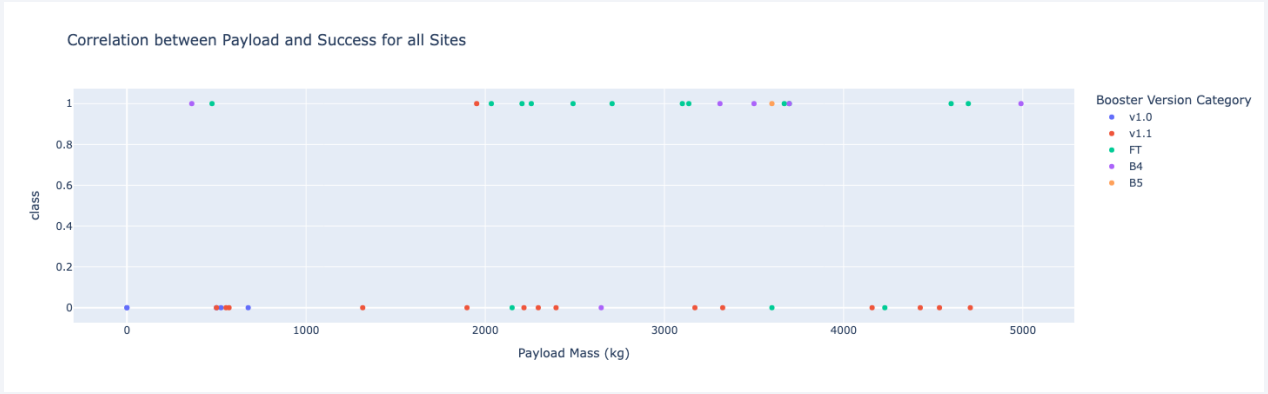
- The pie chart shows the Total Successful Launches by Launch Site. KSC LC-39A has the most successful launches

Most successful Launch Site

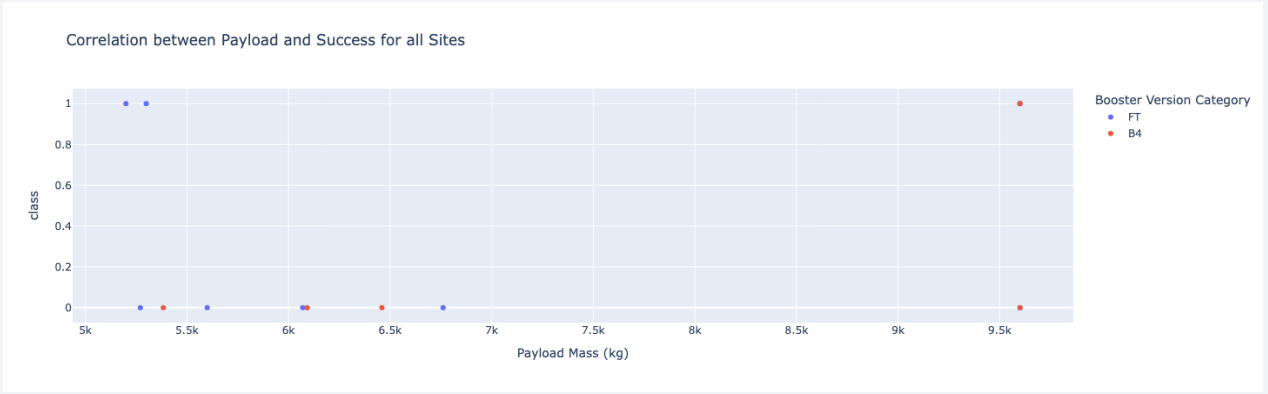


- Pie chart shows the percentages of successful and failed landings in the Most successful Launch Site: KSC LC-39A.
- The site has 76.9% of successful landings and 23.1% of failed landings.

Payload mass vs Outcome for all sites with different payload mass ranges



Payload mass range: 0 to 5000 kg



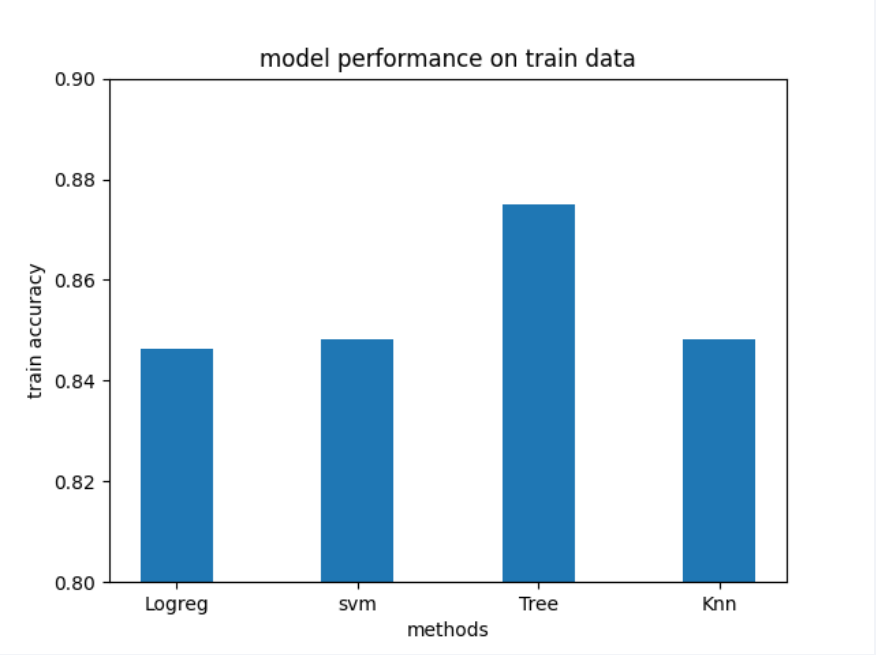
Payload mass range: 5000 to 10000 kg

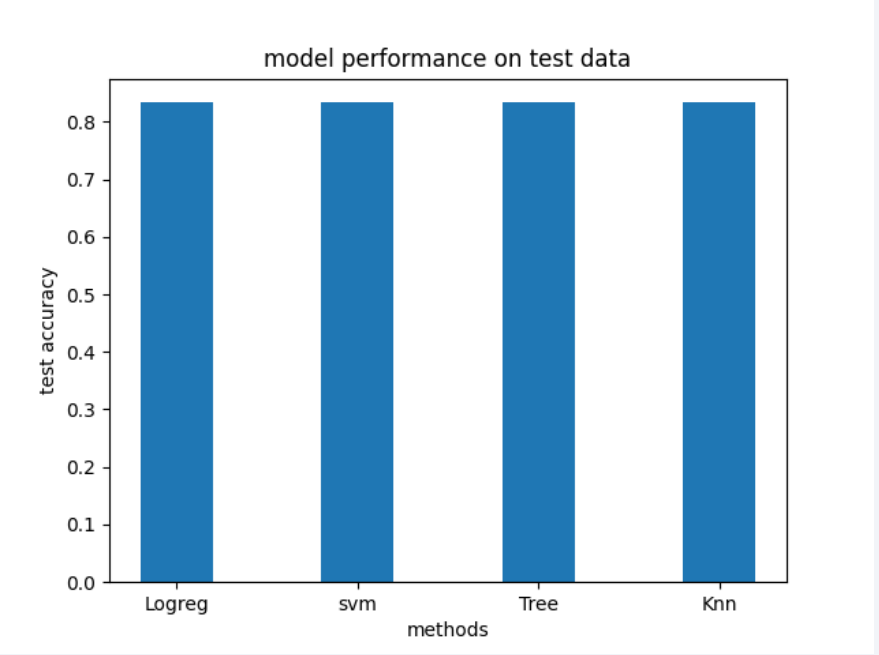
- In the payload range from 0 to 5000 kg, the successful rate is higher. Booster category FT has the highest successful rate.
- In the payload range from 5000 to 10000 kg, the booster category FT has the most successful landings compared to B4. For both booster categories in this payload mass range, the failure rate is higher.

Section 5

Predictive Analysis (Classification)

Classification Accuracy



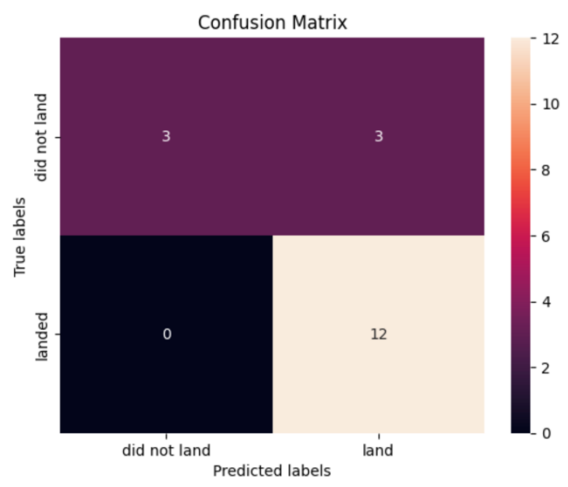


	methods	Accuracy Train	Accuracy Test
0	Logreg	0.846429	0.833333
1	svm	0.848214	0.833333
2	Tree	0.875000	0.833333
3	Knn	0.848214	0.833333

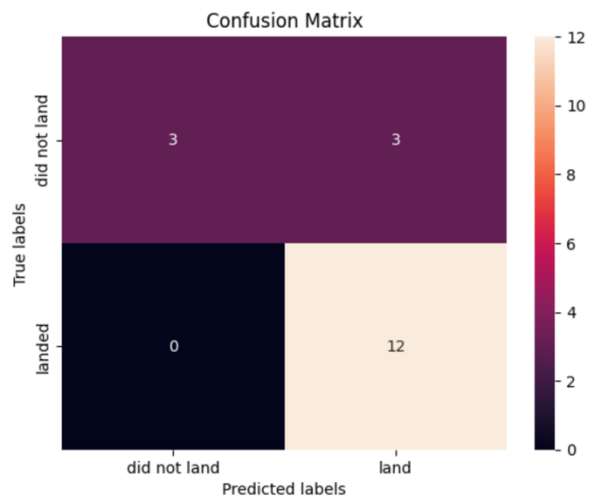
- It is difficult choose the most accurate model using the test data since all the models yielded the same accuracy value of 0.83.
- Based on the accuracies of the train data, **Decision Tree Model** has performed the best with an accuracy of 0.875.

Confusion Matrix

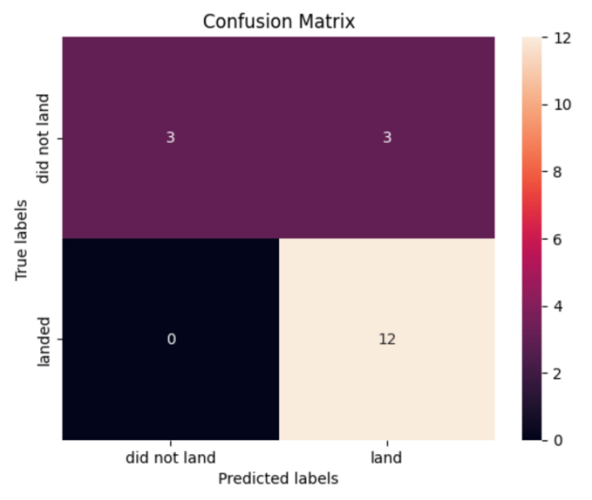
Logistic Regression



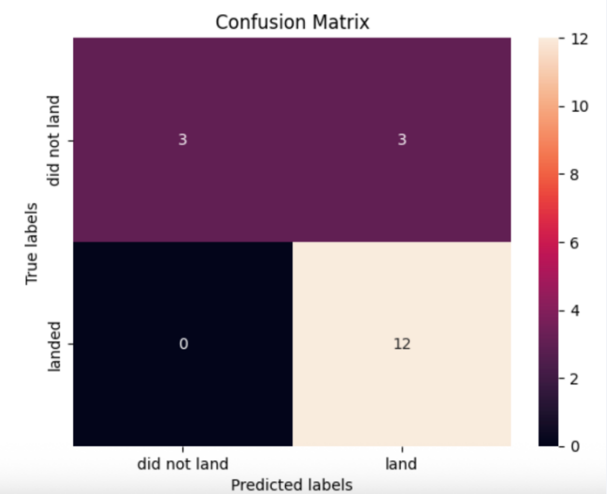
SVM



Decision Tree



KNN



- Since the test accuracy is the same in all the models, the confusion matrix looks identical for all of them.

Conclusions

- As the number of Flight Number increases, the number of successful landings also increase. This shows how the previous launches and experience gained from them has improved the success rate on current launches.
- The orbits ES-L1, GEO, HEO and SSO have the highest success rates.
- The success rates of the launches have been increasing since 2013.
- Launch Site KSC LC-39A has the highest success rate. The reason for this is unclear and additional data and study is required to understand why this site is the most successful.
- The success rate of launches are higher for payloads in the range 0-5000 kg. As the payload mass increases, the success rate drops.
- Decision Tree algorithm is the most accurate in determining the successful and failed landing based on the SpaceX train data. All the algorithms have the same accuracy with the test data so it is difficult to conclude which algorithms performed the best based on the test data set.

Appendix

- All the notebooks, apps and data used for this project are added onto Github which can be accessed via this link: https://github.com/samalkapiy/IBM_Applied_Data_Science_capstone/tree/master

Thank you!